

You can create multiple transitions within a single animation by **defining multiple** properties and steps inside one **@keyframes rule**, or by applying **multiple, comma-separated animations** to a single CSS class.  YouTube +1

Method 1: Multiple Properties within a single @keyframes Rule

The most common way to handle multiple "transitions" (changes) in one animation is to define multiple CSS properties and intermediate percentage stops (keyframes) within a single @keyframes block. The browser will automatically interpolate between these defined points.  OpenClassrooms +3

CSS Example:

CSS

```
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  position: relative;
  animation-name: complexAnimation;
  animation-duration: 4s;
}

@keyframes complexAnimation {
  0% {
    background-color: red;
    left: 0px;
    top: 0px;
  }
  25% {
    background-color: yellow;
    left: 200px;
    top: 0px;
  }
  50% {
    background-color: blue;
    left: 200px;
    top: 200px;
  }
  75% {
    background-color: green;
    left: 0px;
    top: 200px;
  }
  100% {
```

```
background-color: red;
left: 0px;
top: 0px;
}
}
```

In this example, the `background-color`, `left`, and `top` properties all change at specific percentages of the animation duration, creating a complex, multi-step animation. [W3Schools](#)

Method 2: Applying Multiple `@keyframes` Animations to one Class [🔗](#)

You can also apply multiple distinct `@keyframes` rules to a single element by using a comma-separated list in the `animation` shorthand property (or `animation-name` longhand). This allows for layering different, simultaneous effects with different timing functions, delays, and durations if needed. [YouTube +4](#)

CSS Example:

CSS

```
.element {
  /* Apply two animations: 'moveRight' and 'fadeInOut' */
  animation-name: moveRight, fadeInOut;
  animation-duration: 4s, 2s; /* Respective durations */
  animation-timing-function: linear, ease-in-out; /* Respective timing functions */
  animation-iteration-count: infinite, 1; /* Respective iteration counts */
}

@keyframes moveRight {
  from {
    transform: translateX(0);
  }
  to {
    transform: translateX(300px);
  }
}

@keyframes fadeInOut {
  0% {
    opacity: 0;
  }
  50% {
    opacity: 1;
  }
}
```

```
100% {  
  opacity: 0;  
}  
}
```

In this case, the `moveRight` animation runs infinitely, while the `fadeInOut` animation runs only once over its duration. This approach is ideal for combining independent animations. [YouTube +4](#)